



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

# Citation Count	40000
🕒 Difficulty	Intermediate
🏢 Institution	Google
≡ Keyword	ViT, Vision Transformer, Patch Embedding, Self-Attention, Image Classification, Pre-training, Fine-tuning
≡ Research Area	Computer Vision / Image Classification
# Year	2021



이 논문은 NLP의 표준이 된 **Transformer 아키텍처**를 이미지에 직접 적용하여, CNN 없이 순수 Transformer만으로 이미지 분류 task를 수행하는 **ViT (Vision Transformer)**를 제안합니다. 이미지를 고정 크기의 패치로 분할하고 각 패치를 NLP의 토큰처럼 처리합니다. **대규모 데이터(JFT-300M, ImageNet-21k)로 사전 학습**하면 CNN 기반 SOTA를 능가하면서도 훨씬 적은 학습 비용이 드는 것을 보입니다.

1. Introduction

논문이 다루는 분야

이 논문이 다루는 핵심 분야는 **컴퓨터 비전(Computer Vision)**에서의 **이미지 인식(Image Recognition)** — 특히 **이미지 분류(Image Classification)**입니다.

구체적으로는 NLP에서 검증된 **Transformer 아키텍처**를 **수정 없이 이미지에 직접 적용**하는 전략을 탐구합니다.

직관적으로 이해하기: NLP에서 문장을 단어(token) 시퀀스로 처리하듯, ViT는 이미지를 **패치(patch) 시퀀스**로 처리합니다.

- 예: 224×224 이미지 → 16×16 패치 196개 → Transformer 입력
- 각 패치는 NLP의 단어 임베딩처럼 선형 변환 후 Transformer에 입력
- 결과적으로 **이미지를 위한 특별한 설계 없이** 표준 Transformer가 작동

왜 이것이 중요한가?

- CNN은 지역성(locality), 이동 등변성(translation equivariance) 등의 **귀납적 편향(inductive bias)**이 하드코딩되어 있음

- Transformer는 이러한 제약 없이 **데이터에서 직접 패턴을 학습** → 충분한 데이터가 있다면 더 강력
- NLP처럼 비전에서도 "대규모 사전 학습 → 소규모 fine-tuning" 패러다임이 가능함을 처음으로 설득력 있게 입증

해당 task에서 기존 연구 한계점



"왜 기존 컴퓨터 비전은 여전히 CNN에 의존하는가?"

핵심 문제: Transformer를 이미지에 적용하는 naive한 방법은 확장이 불가능합니다.

기존 접근 방식과 각각의 한계:

1. CNN 기반 지배적 아키텍처

ResNet, EfficientNet 등은 이미지 인식의 표준입니다. 하지만:

- 귀납적 편향이 고정되어 있어 유연성 부족
- 파라미터 수와 계산량이 커져도 성능 포화 가능성
- 아키텍처 자체가 "이미지"에 특화되어 NLP 발전의 수혜를 받기 어려움

2. Self-Attention을 이미지에 직접 적용 시 문제

Naive 픽셀 단위 Self-Attention은 픽셀 수의 제곱에 비례하는 계산 비용이 필요합니다.

Attention 비용 $\propto O(n^2)$ where $n = H \times W$

224×224 이미지 → $n = 50,176$ → $n^2 \approx 25$ 억 쌍! **현실적으로 불가능합니다.**

3. 기존 이미지 Transformer 연구의 한계

- Local Self-Attention [Parmar et al., 2018]: 각 픽셀이 로컬 이웃만 attend → 전역 문맥 파악 어려움
- Sparse Transformer [Child et al., 2019]: 근사적 전역 attention → 하드웨어 구현 복잡
- Axial Attention: 행/열 방향으로 분리된 attention → 복잡한 엔지니어링 필요
- 공통 문제: 하드웨어 가속기(TPU/GPU)에서 효율적으로 확장 불가

핵심 통찰: "왜 패치 단위로 처리하면 되는가?"

픽셀 하나하나가 아니라 **16×16 패치 하나**를 하나의 토큰으로 취급하면 시퀀스 길이가 196으로 줄어듭니다 (50,176 → 196). Attention 비용은 $196^2 \approx 38,416$ 으로 **극적으로 감소**합니다. 이 단순한 아이디어가 ViT의 핵심입니다.

논문의 contributions

이 논문의 핵심 기여는 세 가지입니다.

기여 1: 순수 Transformer의 이미지 분류 가능성 입증

CNN 없이 표준 Transformer만으로 이미지 인식이 가능함을 처음으로 대규모로 입증합니다. 이미지 특화 설계 (inductive bias)가 꼭 필요하지 않음을 보입니다.

기여 2: 대규모 사전 학습의 중요성 발견

작은 데이터셋(ImageNet)에서는 CNN보다 낮지만, **대규모 데이터(JFT-300M, 3억 장)로 사전 학습하면 CNN SOTA를 능가**합니다. "대규모 학습이 귀납적 편향을 압도한다"는 핵심 발견입니다.

기여 3: 훨씬 낮은 학습 비용으로 SOTA 달성

BiT-L(ResNet152×4, 9.9k TPuv3-core-days) 대비 ViT-L/16은 **0.68k TPuv3-core-days**로 약 15배 저렴하게 동등하거나 더 나은 성능을 달성합니다.

2. Related Work



논문은 Related Work를 크게 세 방향으로 정리합니다: **NLP Transformer 계보**, **이미지에 Self-Attention 적용 시도들**, **대규모 이미지 인식 연구**입니다.

Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가

1. NLP Transformer 계보

Vaswani et al. (2017) — "Attention is All You Need": Transformer 원논문. 기계번역을 위해 제안되었으나 이후 NLP 전반의 표준이 됨.

BERT [Devlin et al., 2019]: 대규모 비지도 사전 학습 → fine-tuning. ViT는 이 패러다임을 비전으로 이식합니다. [class] 토큰 설계도 BERT에서 직접 차용.

GPT 계열 [Radford et al., 2018; 2019; Brown et al., 2020]: 언어 모델링 사전 학습. GPT-3는 1000억 파라미터 이상으로, Transformer의 극단적 스케일링 가능성을 보임.

2. 이미지에 Self-Attention 적용 시도들

Cordonnier et al. (2020): ViT와 가장 유사한 선행 연구. 2×2 패치로 full self-attention 적용. 하지만 패치가 너무 작아 소해상도 이미지에만 적용 가능. ViT는 16×16/32×32 패치로 중간 해상도까지 확장.

iGPT [Chen et al., 2020]: 픽셀을 직접 토큰으로 사용하는 비지도 Transformer. 해상도를 대폭 줄여야 하며 ImageNet에서 최대 72% 정확도로 ViT보다 낮음.

3. CNN + Self-Attention 하이브리드 연구

Wang et al. (2018), Carion et al. (2020): CNN feature map에 self-attention 추가. ViT는 이 방향을 완전히 벗어나 **pure Transformer**를 추구합니다.

제안 방법의 차별성

비교 대상	Attention 단위	CNN 의존	확장성	사전 학습 규모
ViT (본 논문)	패치 (16×16)	없음	우수	JFT-300M (3억장)
Local SA [Parmar]	픽셀 (로컬)	없음	제한적	소규모
Cordonnier	패치 (2×2)	없음	소해상도만	소규모
CNN+Attention	픽셀/feature	있음	보통	중규모

ViT의 핵심 차별성: **"수정 없는 표준 NLP Transformer + 대규모 사전 학습 → 비전 SOTA"**

3. 제안 방법론

Main Idea



ViT의 핵심 아이디어는 "이미지를 패치 시퀀스로 바꾸면 표준 Transformer를 그대로 쓸 수 있다" 입니다.

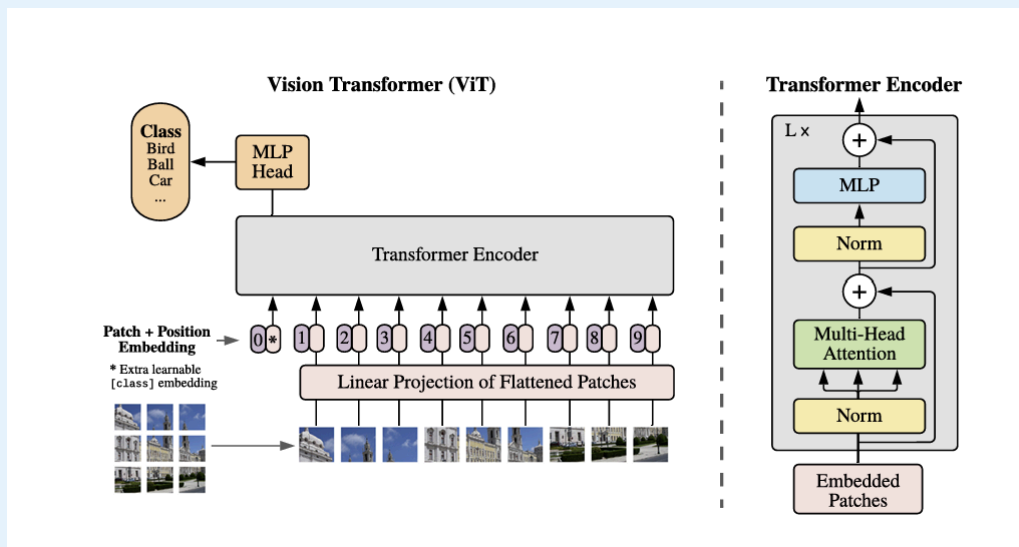
이미지를 16×16 패치로 분할 → 각 패치를 선형 변환 → Transformer Encoder 입력.

전체 과정에서 **이미지 특화 설계는 패치 분할 단계 하나뿐**입니다.

ViT의 전체 파이프라인:

이미지 $x \in \mathbb{R}^{H \times W \times C}$ $\xrightarrow{\text{패치 분할}}$ $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$ $\xrightarrow{\text{선형 투영 } E}$ 패치 임베딩 $\xrightarrow{+ \text{위치 임베딩}}$ Transformer Encoder $\xrightarrow{[\text{CLS}] \text{ 출력}}$ 분류

여기서 $N = HW/P^2$ 는 패치 수 (ViT-B/16 기준: $224^2/16^2 = 196$)



이미지 → 패치 분할 → Linear Projection → [CLS] 토큰 추가 → Positional Embedding → Transformer Encoder → MLP Head → 분류 레이블.

Method 핵심 구성

1) 패치 임베딩 (Patch Embedding)

핵심 수식 4개:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + \mathbf{E}_{\text{pos}}, \quad E \in \mathbb{R}^{(P^2 \cdot C) \times D}, \quad \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \dots L$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \quad \ell = 1 \dots L$$

$$y = \text{LN}(\mathbf{z}_L^0)$$

각 구성 요소의 역할:

- **패치 분할**: $H \times W \times C$ 이미지 → N 개의 $P^2 C$ 차원 패치 벡터. $P=16$ 이면 각 패치는 $16 \times 16 \times 3 = 768$ 차원
- **선형 투영 E**: 각 패치를 D 차원 임베딩으로 변환 (학습 가능한 가중치 행렬)

- **[CLS] 토큰**: BERT의 [CLS]처럼 시퀀스 맨 앞에 추가하는 학습 가능한 임베딩. 최종 Transformer 출력 z_L^0 이 이미지 전체 표현으로 사용됨
- **위치 임베딩 E_{pos}** : 학습 가능한 1D 위치 임베딩. 위치 정보가 없으면 모델이 패치 순서를 알 수 없음

Transformer Encoder 구조:

- Pre-LN (Layer Norm을 블록 앞에 적용): 학습 안정성 향상
- MSA → 잔차 연결 → MLP → 잔차 연결 (Eq. 2, 3)
- MLP는 두 레이어 + GELU 비선형 활성화

2) ViT 모델 변형 (Model Variants)

BERT의 모델 구성을 기반으로 세 가지 크기를 제안합니다.

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

모델	레이어 수 (L)	은닉 크기 (D)	MLP 크기	어텐션 헤드 수	파라미터
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

모델 표기법: ViT-L/16 = Large 변형 + 16×16 패치 크기

- 패치 크기가 작을수록 시퀀스 길이 증가 → 계산 비용 증가
- ViT-B/16 vs ViT-B/32: 같은 파라미터지만 B/16이 훨씬 비쌈

베이스라인 CNN: ResNet (BiT)

- Batch Normalization → Group Normalization 교체
- 표준화된 합성곱(Standardized Convolutions) 사용
- 전이 학습 성능 향상을 위한 수정 (Kolesnikov et al., 2020)

하이브리드 아키텍처:

- CNN feature map을 ViT 입력으로 사용 (패치 크기 = 1픽셀)
- ResNet stage 4 출력 또는 stage 3 확장 출력 사용
- 소규모 모델에서 순수 ViT보다 약간 우수하지만 대규모에서 차이 소실

3) Fine-tuning과 고해상도 처리

Fine-tuning 전략:

- 사전 학습된 예측 헤드 제거 → K개 downstream 클래스를 위한 D×K 레이어 부착
- Fine-tuning 시 **사전 학습보다 높은 해상도** 사용이 유리 (예: 224 → 384 또는 512)

고해상도 처리의 핵심 문제:

- 패치 크기는 고정 → 더 큰 이미지 = 더 긴 시퀀스
- 사전 학습된 위치 임베딩이 새로운 시퀀스 길이와 맞지 않음

해결책: 2D 보간(2D Interpolation)

$$\mathbf{E}_{\text{pos}}^{\text{new}} = \text{Interpolate}_{2D}(\mathbf{E}_{\text{pos}}^{\text{pretrain}}, \text{new_grid_size})$$

원본 이미지에서의 위치에 따라 2D 위치 임베딩을 보간합니다. 이것이 **ViT에서 이미지 2D 구조에 대한 인식이 수동으로 주입되는 유일한 두 번째 지점**입니다 (첫 번째는 패치 추출).

4) 귀납적 편향 (Inductive Bias) 분석

속성	CNN	ViT
지역성 (Locality)	✓ 모든 레이어에 내재	✗ MLP 레이어만 (국소적)
이동 등변성 (Translation Equivariance)	✓ 모든 레이어에 내재	✗ MLP 레이어만
2D 구조 인식	✓ 필터 설계에 내재	⚠ 패치 추출 + 위치 보간 시만
전역 문맥	✗ receptive field 점진적 확대	✓ 모든 레이어에서 전역 attention

핵심 논점: ViT는 귀납적 편향이 적기 때문에 **소규모 데이터에서 CNN보다 나쁘지만**, 대규모 데이터에서는 데이터로부터 직접 관련 패턴을 학습하여 **CNN을 능가**합니다.

4. 실험 및 결과

Dataset

사전 학습 데이터셋 (Pre-training):

데이터셋	클래스 수	이미지 수	특징
ImageNet (ILSVRC-2012)	1k	1.3M	표준 벤치마크, 소규모
ImageNet-21k	21k	14M	공개 대규모 데이터셋
JFT-300M	18k	303M	Google 내부 초대규모, 비공개

평가 데이터셋 (Downstream):

- ImageNet (원본 레이블 + 정제된 ReaL 레이블)
- CIFAR-10/100
- Oxford-IIIT Pets, Oxford Flowers-102
- **VTAB (Visual Task Adaptation Benchmark):** 19개 태스크, 태스크당 1,000개 학습 샘플
 - Natural (7개): Pets, CIFAR 등 일반 이미지
 - Specialized (4개): 의료·위성 이미지
 - Structured (8개): 위치 파악 등 기하학적 이해 필요

Baseline

비교 대상	설명	사전 학습 데이터	TPUv3-core-days
BiT-L (ResNet152×4)	대규모 ResNet 전이 학습 SOTA	JFT-300M	9.9k
Noisy Student (EfficientNet-L2)	반지도학습 기반 ImageNet SOTA	ImageNet+JFT	12.3k
ViT-L/16 (Ours-JFT)	본 논문 중간 크기 모델	JFT-300M	0.68k
ViT-H/14 (Ours-JFT)	본 논문 최대 크기 모델	JFT-300M	2.5k

결과

실험 1: SOTA 비교

주요 이미지 분류 벤치마크 SOTA 비교표.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

주요 수치 (ViT-H/14, JFT-300M 사전학습):

- ImageNet: **88.55%** (vs BiT-L 87.54%, Noisy Student 88.4%)
- ImageNet ReaL: **90.72%**
- CIFAR-100: **94.55%**
- VTAB (19 tasks): **77.63%**
- TPUv3-core-days: **2.5k** (Noisy Student 12.3k 대비 **5배 절약**)

주요 분석:

- JFT-300M 사전학습 ViT-L/16이 BiT-L 대비 **모든 태스크에서 우수**, 학습 비용은 **14.6배 저렴**
- ViT-H/14는 ImageNet, CIFAR-100, VTAB 등 어려운 벤치마크에서 추가 향상
- ImageNet-21k 사전학습 ViT-L/16도 대부분 태스크에서 경쟁력 (표준 TPUv3 8코어로 약 30일)

실험 2: VTAB 세부 분석

VTAB 태스크 그룹별(Natural / Specialized / Structured) 정확도 막대그래프.

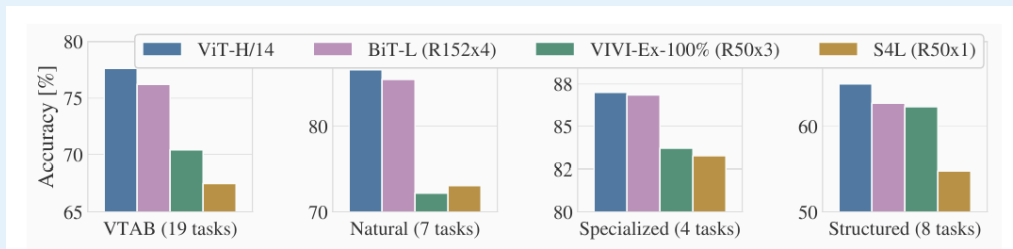


Figure 2: Breakdown of VTAB performance in *Natural*, *Specialized*, and *Structured* task groups.

- **Natural:** ViT-H/14 > BiT-L
- **Structured:** ViT-H/14 > BiT-L (기하학적 이해 필요 태스크에서도 우수)
- **Specialized:** ViT-H/14 ≈ BiT-L (의료·위성 이미지에서 비슷)

실험 3: 사전 학습 데이터 크기 요구량

사전 학습 데이터셋 크기에 따른 ImageNet Top-1 정확도 (ImageNet → ImageNet-21k → JFT-300M).
BiT ResNets(음영) vs 다양한 ViT 변형 비교.

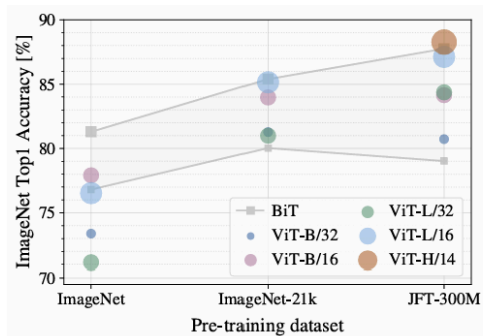


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

JFT 사전 학습 샘플 수(9M~300M)에 따른 ImageNet Linear 5-shot 정확도.

ViT 변형들 vs ResNet(BiT) 비교.

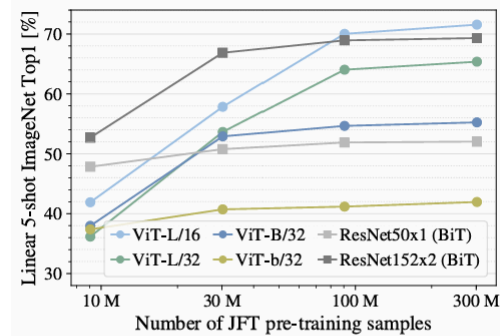


Figure 4: Linear few-shot evaluation on ImageNet versus pre-training size. ResNets perform better with smaller pre-training datasets but plateau sooner than ViT, which performs better with larger pre-training. ViT-b is ViT-B with all hidden dimensions halved.

핵심 발견:

- ImageNet(1.3M)만 사전학습: ViT-Large < ViT-Base (귀납적 편향 부재로 오히려 나쁨)
- ImageNet-21k(14M): ViT-Large ≈ ViT-Base
- JFT-300M(303M): **ViT-Large >> ViT-Base, BiT 능가**
- JFT 9M 이하: ViT-B/32 < ResNet50 (소규모에서 CNN의 귀납적 편향이 유리)
- JFT 90M+: ViT-B/32 > ResNet50 (데이터가 충분하면 Transformer가 역전)

"대규모 학습이 귀납적 편향을 압도한다(Large scale training trumps inductive bias)" — 논문의 핵심 주장

실험 4: 성능 vs 학습 비용

사전 학습 compute(exaFLOPs)에 따른 전이 성능 (Average-5 / ImageNet).

Transformer(ViT) vs ResNet(BiT) vs Hybrid 비교 산점도.

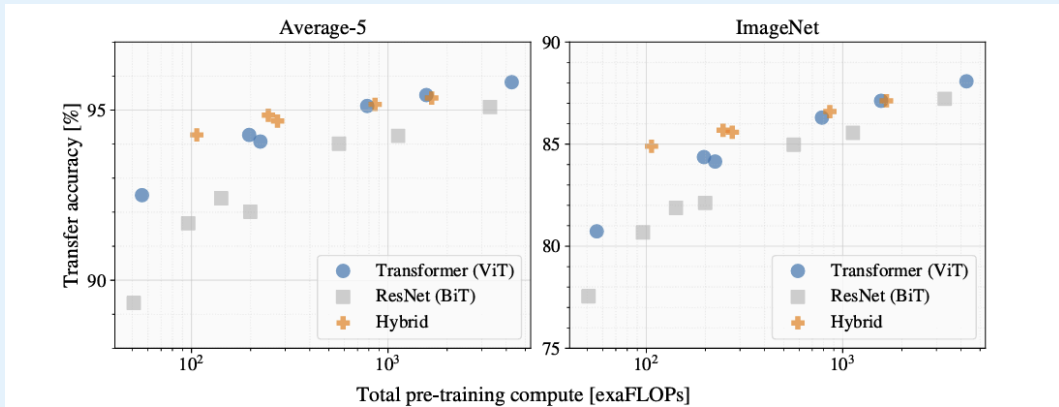


Figure 5: Performance versus pre-training compute for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

핵심 관찰 3가지:

1. ViT가 동일 compute에서 ResNet을 지배: 같은 계산 예산으로 ViT가 2~4배 효율적
2. 소규모에서 Hybrid 우세, 대규모에서 차이 소실: Hybrid(CNN+ViT)가 작은 compute에서 순수 ViT보다 약간 우수하지만 모델이 커지면 차이가 없어짐
3. ViT의 성능 포화 없음: 실험 범위 내에서 스케일 업할수록 계속 향상 → 추가 스케일링 동기

실험 5: ViT 내부 분석

ViT-L/32 내부 표현 분석 3가지.

Left: RGB 임베딩 필터 첫 28개 주성분 / Center: 위치 임베딩 코사인 유사도 히트맵 / Right: 헤드별·레이어별 평균 attention 거리.

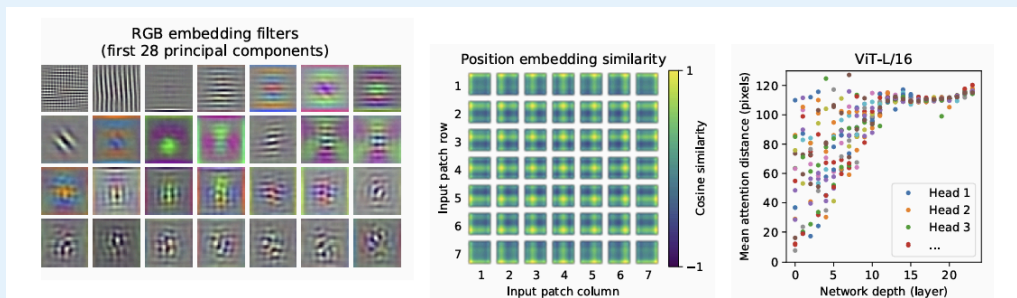


Figure 7: **Left:** Filters of the initial linear embedding of RGB values of ViT-L/32. **Center:** Similarity of position embeddings of ViT-L/32. Tiles show the cosine similarity between the position embedding of the patch with the indicated row and column and the position embeddings of all other patches. **Right:** Size of attended area by head and network depth. Each dot shows the mean attention distance across images for one of 16 heads at one layer. See Appendix [D.7](#) for details.

분석 1 — RGB 임베딩 필터 (Figure 7 Left):

- 첫 번째 선형 투영 레이어가 학습한 필터를 PCA로 시각화
- **Gabor 필터, 색상 필터 등 의미 있는 저수준 특징**이 자동으로 학습됨
- CNN의 초기 레이어 필터와 유사 → Transformer도 유사한 저수준 특징을 학습

분석 2 — 위치 임베딩 유사도 (Figure 7 Center):

- 각 패치 위치의 위치 임베딩 간 코사인 유사도를 시각화
- **가까운 패치일수록 유사한 위치 임베딩**을 가짐 (2D 토폴로지 학습!)
- 행/열 구조가 명확히 나타남 → 1D 위치 임베딩만 써도 2D 구조를 스스로 학습

분석 3 — Attention 거리 (Figure 7 Right):

- 헤드별, 레이어별 평균 attention 거리 (CNN의 receptive field에 해당)
- **가장 낮은 레이어에서 이미 일부 헤드가 이미지 전체를 attend** (전역 문맥 처리가 처음부터 가능)
- 다른 헤드들은 낮은 레이어에서 작은 attention 거리 유지 (로컬 처리)
- 레이어 깊어질수록 모든 헤드의 attention 거리 증가

출력 토큰에서 입력 공간으로의 attention 시각화 대표 예시.

개, 비행기, 꽃 이미지에서 의미론적으로 관련된 영역에 attend함.

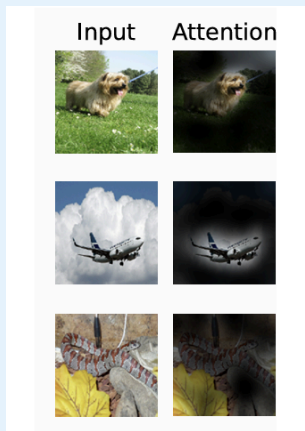


Figure 6: Representative examples of attention from the output token to the input space. See Appendix [D.7](#) for details.

5. Ablation Studies (소거 실험)

위치 임베딩 방식 비교 (Appendix D.4, Table 8)

ViT-B/16, ImageNet 5-shot linear 평가.

Pos. Emb.	Default/Stem	Every Layer	Every Layer-Shared
No Pos. Emb.	0.61382	N/A	N/A
1-D Pos. Emb.	0.64206	0.63964	0.64292
2-D Pos. Emb.	0.64001	0.64046	0.64022
Rel. Pos. Emb.	0.64032	N/A	N/A

Table 8: Results of the ablation study on positional embeddings with ViT-B/16 model evaluated on ImageNet 5-shot linear.

위치 임베딩 방식	ImageNet 5-shot (Default)	핵심 결과
위치 임베딩 없음	0.6138	성능 대폭 하락 → 위치 정보 필수
1D 위치 임베딩 (기본)	0.6421	기본값, 충분히 우수
2D 위치 임베딩	0.6400	1D와 거의 동일
상대적 위치 임베딩	0.6403	1D와 거의 동일

결론: 위치 임베딩 종류보다 **있고 없음이 훨씬 중요**. 패치 단위라 공간 해상도가 낮아(14×14) 복잡한 2D 임베딩의 이점이 없음.

[CLS] 토큰 vs Global Average Pooling (Appendix D.3, Figure 9)

CLS 토큰 방식 vs GAP 방식의 ImageNet linear 5-shot 학습 곡선 비교.

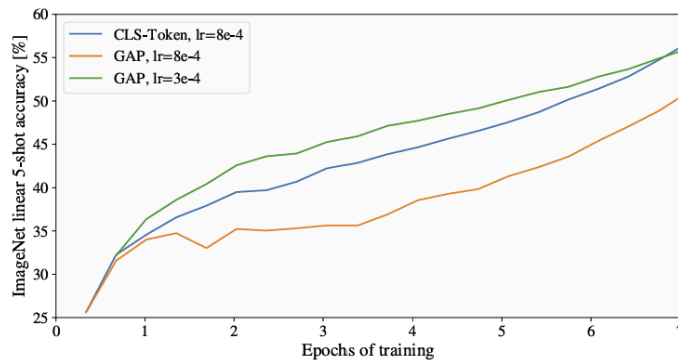
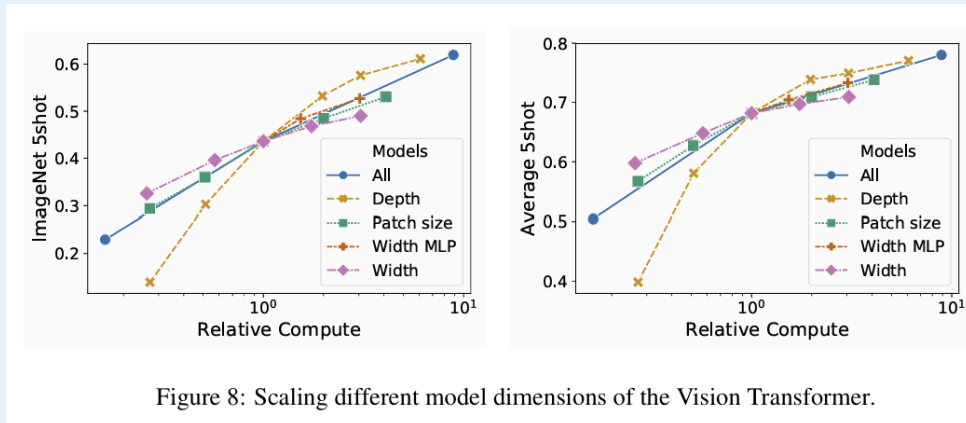


Figure 9: Comparison of class-token and global average pooling classifiers. Both work similarly well, but require different learning-rates.

- [CLS] 토큰과 GAP 모두 최종 성능은 유사
- 차이는 **최적 학습률**이 다르다는 것뿐 (CLS: lr=8e-4, GAP도 lr=3e-4면 동일)
- 논문은 NLP Transformer와의 일관성을 위해 [CLS] 토큰을 채택

Transformer 스케일링 방향 비교 (Appendix D.2, Figure 8)

Depth / Patch size / Width MLP / Width 각 차원별 스케일링 효과 (ImageNet 5-shot, Average 5-shot).



- **Depth 스케일링 효과 최대:** 64레이어까지 명확한 향상
- **Width 스케일링 효과 최소**
- **패치 크기 감소:** 파라미터 추가 없이 robust한 향상
- **결론:** compute 대비 성능 향상에는 **depth > patch size > width**

Self-supervised 사전 학습 예비 실험 (Section 4.6)

BERT의 MLM과 유사한 **Masked Patch Prediction** 실험:

- 패치 임베딩의 50%를 랜덤 마스킹 (BERT와 동일한 80/10/10 전략)
- 마스킹된 패치의 평균 3bit 색상 예측
- ViT-B/16: self-supervised 79.9% vs supervised 83.9% → 여전히 4% 격차
- 스크래치 학습 대비 2% 향상
- **결론:** 자기지도 학습이 가능성을 보이나 지도 학습과의 격차가 여전히 큼 (이후 MAE 등으로 발전)

6. 결론 (배운점)

이 논문의 가장 큰 의의는 **"이미지에 CNN이 필수가 아니다 — 충분한 데이터와 표준 Transformer만으로 비전 SOTA를 달성할 수 있다"**는 것을 설득력 있게 입증한 것입니다.

ViT 이후 컴퓨터 비전 연구의 방향이 크게 바뀌었습니다.

"어떻게 CNN을 개선할 것인가" 에서 **"어떻게 Transformer를 비전에 최적화할 것인가"** 로 전환.

ViT는 Swin Transformer, DeiT, MAE, CLIP, DALL-E, SAM의 직접적인 토대입니다.

특히 **"대규모 사전 학습 → fine-tuning"** 패러다임이 비전에도 정착되는 결정적 계기가 됩니다.

ViT가 가르쳐 준 설계 철학:

- **단순한 패러다임:** CNN-specific 설계 없이 NLP 도구를 그대로 쓰는 것이 최고 성능

- **데이터가 귀납적 편향을 대체:**
 - 3억 장 규모의 데이터가 있으면, CNN처럼 구조적으로 넣어주던 inductive bias 없이도 모델이 스스로 학습한다.
 - → 충분한 데이터가 있으면 구조보다 데이터가 더 중요해진다.
- **스케일링은 depth 중심으로:** width보다 depth, 파라미터보다 compute
 - → 파라미터 수 자체보다, 얼마나 깊게 쌓느냐가 더 중요하다.
- **위치 임베딩의 유연성:** 1D 임베딩으로도 2D 구조를 학습 가능
 - → 굳이 복잡한 2D 구조를 강제로 넣지 않아도 된다.

7. 느낀점

좋았던 점

1. 단순함으로 패러다임을 바꾼 논문

"이미지를 패치 시퀀스로 보면 된다"는 아이디어는 지금 보면 굉장히 당연해 보인다. 하지만 이걸 실제 모델로 구현하고, 대규모 실험으로 검증했다는 점이 핵심 기여다.

2. 데이터 크기에 따른 체계적인 분석

ImageNet → ImageNet-21k → JFT-300M으로 데이터 규모를 점점 키워가며 성능 변화를 분석한다.

어떤 조건에서 좋아지고, 언제 한계가 드러나는지를 명확하게 보여주는 실험 설계가 인상적이다.

3. 한계를 인정

소규모 데이터에서는 CNN보다 성능이 떨어진다는 점, JFT-300M이 비공개라 재현이 어렵다는 점 등을 숨기지 않고 명확하게 언급한다.

4. 내부 동작을 이해하려는 시도

Figure 7에서 필터 시각화, 위치 임베딩 유사도, attention 거리 등을 분석한다. 단순히 결과만 보여주는 것이 아니라 모델이 실제로 무엇을 학습하는지, 왜 잘 작동하는지를 이해하려는 노력이 보인다.

아쉬웠던 점

1. JFT-300M 비공개로 인한 재현성 문제

최고 성능 결과가 Google 내부 데이터셋(JFT-300M)에 기반한다. 이 때문에 대부분의 연구자가 동일한 조건에서 결과를 재현하기 어렵다.

ImageNet-21k에서도 좋은 성능을 보이긴 하지만, JFT 결과와의 격차가 커서 논문의 핵심 주장까지 완전히 검증하기는 어렵다고 느껴진다.

2. 분류 외 비전 태스크 확장 부족

논문이 이미지 분류에만 집중되어 있다. 객체 탐지나 의미론적 분할 같은 다른 태스크에서 어떻게 동작하는지는 다루지 않는다. 이 때문에 ViT의 실제 활용 범위를 바로 판단하기는 어렵다.

3. 소규모 데이터에서의 성능 저하 대응 부족

ViT가 작은 데이터에서 CNN보다 성능이 떨어지는 이유는 설명한다. 하지만 이를 해결하기 위한 방법은 비교적 기본적인 수준에 머문다. 이후 연구들처럼 더 적극적인 개선 방향까지 제시했으면 좋았을 것 같다.

4. Hybrid 구조에 대한 분석 부족

Hybrid(CNN + ViT)가 작은 compute 환경에서는 더 좋은 성능을 보인다. 하지만 왜 그런지에 대한 깊은 분석은 부족하다.

또한 데이터와 compute가 커질수록 이 차이가 왜 사라지는지도 직관적으로 설명되지 않는다.



You Only Look Once: Unified, Real-Time Object Detection

# Citation Count	40000
🕒 Difficulty	Intermediate
🏛️ Institution	Academic
≡ Keyword	YOLO, Object Detection, Real-Time, CNN, Bounding Box, Regression, Grid Cell
≡ Research Area	Computer Vision / Object Detection
# Year	2016



이 논문은 객체 탐지를 **단일 회귀 문제**로 재정의하여, 이미지 전체를 **한 번만 보고(You Only Look Once)** 바운딩 박스와 클래스 확률을 동시에 예측하는 **YOLO (You Only Look Once)**를 제안합니다. 기존의 복잡한 파이프라인(R-CNN 계열)과 달리 하나의 CNN이 전체 탐지 과정을 end-to-end로 수행하며, 45 FPS(Fast YOLO는 155 FPS)의 실시간 처리 속도와 기존 실시간 탐지기 대비 2배 이상의 mAP를 달성합니다.

1. Introduction

논문이 다루는 분야

이 논문이 다루는 핵심 분야는 **컴퓨터 비전(Computer Vision)**에서의 **객체 탐지(Object Detection)**입니다.

객체 탐지는 이미지 내에서 **"무엇이"** 있는지(분류)와 **"어디에"** 있는지(위치)를 동시에 예측하는 task입니다. 자율주행, 보조 기기, 로봇공학 등 실제 응용에서 핵심적인 역할을 합니다.

직관적으로 이해하기: 인간은 이미지를 한 번 훑어 보는 것만으로도 어떤 물체가 어디에 있는지 즉시 파악합니다. YOLO는 이 과정을 모방합니다:

- **기존 방식:** 이미지를 여러 부분으로 나눠 각각을 classifier에 통과시키는 다단계 파이프라인
- **YOLO 방식:** 이미지 전체를 단 한 번의 신경망 추론으로 처리

해당 task에서 기존 연구 한계점



"왜 기존 객체 탐지 시스템은 느리고 복잡한가?"

핵심 문제: 탐지를 분류 문제로 우회하는 복잡한 다단계 파이프라인입니다.

기존 접근 방식과 각각의 한계:

1. Sliding Window 방식: DPM (Deformable Parts Models)

DPM은 고정된 슬라이딩 윈도우로 이미지 전체를 스캔하며 classifier를 반복 실행합니다.

문제점:

- 이미지 전체에 걸쳐 classifier를 수천 번 반복 실행 → 매우 느림
- 특징 추출, 영역 분류, 바운딩 박스 예측이 각각 분리된 파이프라인 → 최적화 어려움
- 정적(static) 특징을 사용하여 표현력이 제한적

2. 영역 제안(Region Proposal) 방식: R-CNN 계열

R-CNN은 Selective Search로 ~2,000개의 후보 영역을 먼저 생성한 뒤, 각 영역에 CNN을 적용합니다.

문제점:

- 테스트 시 이미지 하나당 40초 이상 소요 → 실시간 불가능
- Selective Search, CNN 특징 추출, SVM 분류, 바운딩 박스 회귀, NMS 등 각 단계를 독립적으로 학습 → end-to-end 최적화 불가
- Fast R-CNN, Faster R-CNN으로 개선되었으나 여전히 실시간 수준에 미달 (0.5~7 FPS)

핵심 통찰: "탐지를 왜 분류 문제로 풀려 하는가?"

탐지는 본질적으로 회귀(regression) 문제입니다. 이미지 픽셀에서 바로 좌표와 클래스 확률을 예측하면 됩니다. 우회로(분류기 → 후처리)를 거칠 필요가 없습니다.

논문의 contributions

이 논문의 핵심 기여는 세 가지입니다.

기여 1: 탐지를 단일 회귀 문제로 통합

이미지 픽셀에서 바운딩 박스 좌표와 클래스 확률을 하나의 신경망이 직접 예측합니다. 복잡한 다단계 파이프라인을 완전히 제거합니다.

기여 2: 압도적인 실시간 처리 속도

Base YOLO 45 FPS, Fast YOLO 155 FPS로, 기존 실시간 탐지기(30Hz DPM) 대비 2배 이상의 mAP를 달성합니다. 25ms 미만의 latency로 스트리밍 비디오를 실시간 처리합니다.

기여 3: 강력한 일반화 성능

자연 이미지로 학습하고 예술 작품에서 테스트해도 DPM, R-CNN 대비 월등히 높은 일반화 성능을 보입니다.